

MERIDIAN TRUST

Web Security Training Handbook

STUDENT EDITION

OWASP Top 10 Penetration-Testing Lab
Setup · Testing Methodology · Challenges · Worksheet

 **INTENTIONALLY VULNERABLE — TEACHING USE ONLY — DO NOT DEPLOY**



Discover and exploit each objective yourself.
Payloads, exploitation steps, and fixes are withheld — ask your instructor for the solution edition.

Contents

Part

1 Setup & Safety

Part

2 Testing Methodology & Proxy Setup

Part

3 Challenge Objectives (find & exploit)

Part

4 Worksheet & Grading

PART 1

Setup & Safety

⚠ **INTENTIONALLY VULNERABLE.** Never deploy to a public network, shared host, or the internet. Run only on an isolated lab machine / VM bound to localhost.

Run with Docker (recommended)

```
cd DefendShield-VulnBank
docker compose up --build
# App: http://127.0.0.1:3000 (bound to 127.0.0.1 only)
```

Docker gotcha on Kali/Linux (`docker-credential-desktop` error)

If `docker compose build` fails with `error getting credentials ... docker-credential-desktop ... not found`, your `~/.docker/config.json` references the Docker Desktop credential helper. Fix by removing the `credsStore` line:

```
# edit ~/.docker/config.json and delete the line: "credsStore": "desktop",
# (or, quick one-liner:)
sed -i '/"credsStore": "desktop"/d' ~/.docker/config.json
docker compose up --build
```

Run with Node directly

```
cd DefendShield-VulnBank
npm install
npm start # http://127.0.0.1:3000
PORT=3001 npm start # use another port if 3000 is busy
```

Verify the lab (API test harness)

An automated harness exercises each planted vulnerability over HTTP and reports PASS/FAIL — useful for regression testing after changes and as API-testing examples for students.

```
npm test # tests http://127.0.0.1:3001 by default
BASE=http://127.0.0.1:3000 npm test # target a different port
```

Run `npm run reseed` and restart first for a clean, repeatable run (some tests mutate data & pollute the process). A clean run reports `101/101 automated passed, 0 failed, 14 manual/skipped`. The 14 skips are client-side (DOM XSS, tabnabbing, `postMessage`, DOM clobbering, `localStorage`), browser-CSRF, or DoS/timing checks (ReDoS, race condition, unbounded body, time-based SQLi) that must be verified by hand — the harness lists each with a reason.

Reset the lab between students

```
npm run reseed # wipes db/vulnbank.sqlite and re-seeds fake data
```

Seed accounts

Username	Password	Role
admin	admin	admin
support	support123	support
alice	password1	user
bob	hunter2	user
carol	letmein	user

Proxy tooling

Configure Burp Suite or OWASP ZAP as an HTTP proxy on `127.0.0.1:8080` and point your browser at it. Full step-by-step exploitation is in [FINDINGS.md](#). Recommended Burp extensions: **JWT Editor**, **Turbo Intruder**, **Logger++**.

Safety rails built in

- Binds to `127.0.0.1` in `docker-compose.yml`.
- Loud red "VULNERABLE" banner on every page.
- Only fake, seeded data — no real PII.
- `reseed` script for a clean, repeatable classroom state.

PART 2

Testing Methodology & Proxy Setup

Burp Suite (Community/Pro) 1. Proxy → Options → Proxy Listeners: confirm `127.0.0.1:8080` is running. 2. Browser: set HTTP/HTTPS proxy to `127.0.0.1:8080` (use FoxyProxy or Burp's built-in browser: Proxy → Open Browser). 3. Proxy → Intercept → **Intercept is on** to catch/modify requests; use **HTTP history** to review, and **Send to Repeater** (Ctrl+R) to replay/tamper. 4. Since the app is plain HTTP on localhost, no CA cert import is needed.

OWASP ZAP 1. Tools → Options → Local Proxies: `127.0.0.1:8080`. 2. Point the browser at it; browse the app to populate the **Sites** tree. 3. Right-click a request → **Open/Resend with Request Editor** to tamper, or **Attack** → **Fuzz** for automated payloads.

General workflow for every finding: browse the feature normally with intercept off to record a baseline request in HTTP history → **Send to Repeater** → modify the parameter → **Send** → observe the response.

PART 3

Challenge Objectives

How to use this edition. Each objective names a vulnerability class and where to start looking. The *how* — payloads, exploitation steps, and remediation — is intentionally omitted. Use your proxy (Burp Suite / OWASP ZAP), test methodically, and record your evidence in the worksheet.

A01 — Broken Access Control

ID	Objective — find & exploit	Where to look	Done
VB-001	IDOR — read any account	GET /account/:id	☐
VB-002	IDOR — any transaction	GET /transaction/:id	☐
VB-003	/ VB-007 Mass assignment → privilege escalation	POST /profile/:id	☐
VB-005	Broken function-level authorization — admin panel	GET /admin	☐
VB-006	Forced browsing	/admin	☐
VB-008	/ VB-075 Horizontal esc + negative-amount transfer (business logic)	POST /transfer	☐
VB-009	/ VB-202 Path traversal / LFI	GET /download?file=	☐
VB-012	/ VB-013 CORS misconfiguration	src/server.js	☐

A02 — Cryptographic Failures

ID	Objective — find & exploit	Where to look	Done
VB-055	Plaintext passwords + VB-058 sensitive data exposure	src/db.js	☐
VB-059	Weak JWT secret	POST /api/login	☐
VB-014	JWT alg:none bypass	auth()	☐
VB-063	Insecure session cookie	express-session	☐
VB-065	Predictable reset token + VB-158 host-header injection	POST /reset	☐

A03 — Injection

ID	Objective — find & exploit	Where to look	Done
VB-021	SQL injection — login bypass	POST /login	☐
VB-022	UNION SQLi — data extraction	GET /search?q=	☐
VB-030	Stored XSS + VB-031 Reflected XSS	memo	☐
VB-040	OS command injection	POST /admin/ping	☐
VB-043	Server-Side Template Injection (SSTI)	display_name	☐
VB-161	XXE — external entity file read	POST /import/xml	☐
VB-045	CRLF / header injection (runtime-mitigated — teaching note)	GET /setlang?lang=	☐

Injection variants

ID	Objective — find & exploit	Where to look	Done
VB-044	NoSQL / object injection — login bypass	POST /nosql-login	☐
VB-023	boolean-blind & VB-024 time-based SQLi	GET /blind?id=	☐
VB-025	second-order SQLi	POST /so/set-nick	☐
VB-026	header-based SQLi	GET /track	☐
VB-053	ReDoS	GET /validate?email=	☐

JWT attack suite

ID	Objective — find & exploit	Where to look	Done
VB-062	Algorithm confusion (RS256 → HS256)	GET /jwt/rs256/whoami	☐
VB-060	kid header injection	GET /jwt/kid/whoami	☐

GraphQL (/graphql, GraphiQL enabled)

ID	Objective — find & exploit	Where to look	Done
VB-175	introspection + VB-182 excessive exposure + VB-179 BOLA + VB-178 no field authz	– discover –	☐

A08 Integrity

ID	Objective — find & exploit	Where to look	Done
	VB-130/131 Insecure deserialization → RCE	GET /integrity/prefs/load?prefs=	☐
VB-196	Zip Slip	POST /integrity/import-zip	☐
VB-135	CSV / formula injection	GET /integrity/export.csv	☐

A02 Weak crypto (extra)

ID	Objective — find & exploit	Where to look	Done
VB-068	AES-ECB pattern leakage	GET /crypto/ecb?text=	☐
VB-072	Reversible "encryption" token	GET /crypto/token	☐

A07 OTP / MFA

ID	Objective — find & exploit	Where to look	Done
VB-098	client-side OTP + VB-099 brute + VB-100 reuse	GET /otp/mfa/send	☐

A01/Client-side (extra)

ID	Objective — find & exploit	Where to look	Done
VB-173	Cross-Site WebSocket Hijacking + VB-174 missing message authz	ws://127.0.0.1:3000/ws	☐

Injection (more)

ID	Objective — find & exploit	Where to look	Done
VB-049	SSI injection	GET /ssi?tpl=	☐
VB-048	LDAP injection	GET /ldap?user=	☐
VB-029	Stacked-query SQLi	GET /note?data=	☐
VB-028	ORDER BY / column-name SQLi	GET /sortusers?order=	☐
VB-042	Argument injection (option smuggling)	GET /archive?path=	☐

Business logic (more)

ID	Objective — find & exploit	Where to look	Done
VB-172	HTTP Parameter Pollution	GET /pay?amount=	☐
VB-104	Password change without current password	POST /account/password	☐
VB-085	Client-trusted price	POST /shop/buy	☐
VB-086	Voucher/gift-card enumeration	GET /voucher?code=	☐
VB-080	Currency rounding abuse	GET /fx?amount=	☐

Integrity / crypto (more)

ID	Objective — find & exploit	Where to look	Done
VB-136	Unverified webhook signature	POST /integrity/webhook/receive	☐
VB-071	Hash length-extension	GET /crypto/sign	☐

OAuth

ID	Objective — find & exploit	Where to look	Done
VB-103	Unvalidated redirect_uri + VB-102 missing state	GET /oauth/authorize	☐

Web cache & content-type

ID	Objective — find & exploit	Where to look	Done
VB-159	Cache poisoning (unkeyed header)	GET /home-banner	☐
VB-160	Cache deception	GET /account-info/*.css	☐
VB-165	Content-type sniffing	GET /raw?body=	☐

Client-side (GET /client — use browser DevTools)

ID	Objective — find & exploit	Where to look	Done
VB-070	Padding oracle (AES-CBC)	GET /crypto/po/token	☐
VB-046	Log injection / forged entries	GET /track-event?name=	☐
VB-050	SMTP / email header injection	GET /contact?subject=	☐
VB-047	XPath injection	GET /xpath?user=	☐
VB-194	Expensive query (DoS)	GET /report?n=	☐
VB-145	SSRF non-HTTP scheme + VB-069 blocklist bypass	GET /proxy?url=	☐
VB-034	Reflected XSS in 404 page	src/server.js	☐
VB-035	Referer-reflected XSS + VB-038 JSON served as text/html	GET /welcome	☐
VB-154	CSRF via GET + VB-155 JSON CSRF	GET /quick-transfer? to=&amount=	☐
VB-092	Timing-based username enumeration	GET /user-check?u=	☐
VB-198	EXIF metadata not stripped	POST /upload	☐
VB-206	/ VB-207 GraphQL mutations without authorization	POST /graphql	☐
VB-208	Broken function-level authz via HTTP verb (no-auth PUT)	PUT /user/:id	☐
VB-209	IDOR via predictable account number	GET /api/acct?num=	☐
VB-210	Reflected File Download (RFD)	GET /statement/export?name=	☐
VB-211	Stored XSS via username (admin widget)	GET /admin/recent	☐
VB-212	Prototype pollution → privilege escalation	POST /profile/:id	☐
VB-213	Billion-laugh / XML entity-expansion DoS	POST /import/xml2	☐
VB-214	Forgeable "remember me" cookie → impersonation	GET /remember?user=	☐

PART 4

Worksheet & Grading

Student worksheet (fill in as you go)

For each finding: record the **request** (from proxy history), the **payload**, the **evidence** (response snippet/screenshot), the **impact**, and the **remediation**.

#	OWASP	Vulnerability	Endpoint	Payload used	Evidence	Impact	Fix
VB-001	A01	IDOR - account	/account/:id				
VB-007	A01	Mass assignment	POST /profile/:id				
VB-005	A01	Missing func-level authz	/admin				
VB-009	A01	Path traversal	/download?file=				
VB-021	A03	SQLi login bypass	POST /login				
VB-022	A03	UNION SQLi	/search?q=				
VB-030	A03	Stored XSS	POST /transfer memo				
VB-040	A03	Command injection	POST /admin/ping				
VB-043	A03	SSTI	/greet				
VB-161	A03	XXE	POST /import/xml				
VB-055	A02	Plaintext passwords	/api/v1/users				
VB-059	A02	Weak JWT secret	POST /api/login				
VB-014	A02	JWT alg:none	/api/users				
VB-065	A02	Predictable reset token	POST /reset				
VB-075	A04	Negative transfer	POST /transfer				
VB-077	A04	Race condition	POST /transfer				
VB-106	A05	Debug/env exposure	/debug				
VB-114	A05	Unrestricted upload	POST /upload				
VB-124	A06	Prototype pollution	POST /profile/:id				
VB-087	A07	Brute force / no lockout	POST /login				
VB-101	A07	Header-trust auth	any + X-User-Id				
VB-138	A10	SSRF avatar-by-URL	/avatar/fetch?url=				
VB-140	A10	SSRF webhook	POST /webhook				
VB-044	A03	NoSQL/object injection	POST /nosql-login				
VB-023	A03	Blind/time SQLi	/blind?id=				
VB-025	A03	Second-order SQLi	/so/set-nick , /so/ lookup				
VB-026	A03	Header SQLi	/track (User-Agent)				

#	OWASP	Vulnerability	Endpoint	Payload used	Evidence	Impact	Fix
VB-053	A03	ReDoS	/validate?email=				
VB-062	A02	JWT alg confusion	/jwt/rs256/whoami				
VB-060	A02	JWT kid injection	/jwt/kid/whoami				
VB-175	A03	GraphQL introspection/BOLA	/graphql				
VB-130	A08	Insecure deserialization	/integrity/prefs/load				
VB-196	A08	Zip Slip	POST /integrity/import-zip				
VB-135	A08	CSV formula injection	/integrity/export.csv				
VB-068	A02	AES-ECB leakage	/crypto/ecb				
VB-072	A02	Reversible token	/crypto/whoami				
VB-098	A07	OTP bypass/brute	/otp/mfa/*				
VB-173	A01	WebSocket hijacking	ws:///ws				
VB-049	A03	SSI injection	/ssi?tpl=				
VB-048	A03	LDAP injection	/ldap?user=				
VB-029	A03	Stacked-query SQLi	/note?data=				
VB-028	A03	ORDER BY SQLi	/sortusers?order=				
VB-042	A03	Argument injection	/archive?path=				
VB-172	A04	HTTP Parameter Pollution	/pay?amount=				
VB-104	A04	Password change no current	POST /account/password				
VB-085	A04	Client-trusted price	POST /shop/buy				
VB-086	A04	Voucher enumeration	/voucher?code=				
VB-080	A04	Currency rounding abuse	/fx?amount=				
VB-136	A08	Unverified webhook sig	POST /integrity/webhook/receive				
VB-071	A02	Hash length-extension	/crypto/sign , /crypto/verify				
VB-103	A01	OAuth redirect_uri/state	/oauth/authorize				

#	OWASP	Vulnerability	Endpoint	Payload used	Evidence	Impact	Fix
VB-159	A05	Cache poisoning	/home-banner (X-Forwarded-Host)				
VB-160	A05	Cache deception	/account-info/*.css				
VB-165	A05	Content-type sniffing	/raw?body=				
VB-032	A03	DOM XSS	/client#...				
VB-157	A05	Reverse tabnabbing	/client				
VB-166	A05	postMessage no origin	/client				
VB-167	A05	DOM clobbering	/client				
VB-169	A02	Secrets in localStorage	/client				
VB-070	A02	Padding oracle	/crypto/po/*				
VB-046	A09	Log injection	/track-event , /logs				
VB-050	A03	SMTP header injection	/contact				
VB-047	A03	XPath injection	/xpath?user=				
VB-194	A04	Expensive-query DoS	/report?n=				
VB-145	A10	SSRF file:// + bypass	/proxy?url=				
VB-034	A03	404 reflected XSS	/<payload>				
VB-035	A03	Referer XSS	/welcome				
VB-038	A03	JSON-as-HTML XSS	/api-echo?q=				
VB-154	A01	CSRF via GET	/quick-transfer				
VB-092	A07	Timing user enum	/user-check?u=				
VB-198	A05	EXIF not stripped	/exif?file=				
VB-206	A01	GraphQL mutation no authz	POST /graphql				
VB-208	A01	No-auth PUT mass assign	PUT /user/:id				
VB-209	A01	IDOR by account number	/api/acct?num=				
VB-210	A05	Reflected File Download	/statement/export?name=				
VB-211	A03		/admin/recent				

#	OWASP	Vulnerability	Endpoint	Payload used	Evidence	Impact	Fix
		Stored XSS via username					
VB-212	A06	Proto-pollution privesc	/profile → /feature/flags				
VB-213	A03	Billion-laugh XML bomb	POST /import/xml2				
VB-214	A07	Forgeable remember-me	/remember?user=				

Grading rubric (suggested)

- **Discovery** (found it via proxy): 40%
- **Exploitation** (working PoC): 40%
- **Remediation** (correct fix described): 20%